



Auto-Approve WebFetch And WebSearch In Claude Code With Hooks

Technology

📅 December 26, 2025 👤 Admin 💬 Leave A Comment

TL;DR: Claude Code’s standard permission settings for WebFetch and WebSearch have known bugs. Hooks bypass these issues and work in CLI, VSCode, and sub-agents.

If you use Claude Code regularly, you’ve likely encountered the permission prompt fatigue. Every web search, every URL fetch: “Allow WebSearch?” “Allow WebFetch?” Click. Click. Click. You add these tools to your settings.local.json allow list, and yet the prompts keep appearing.

It’s a collection of known bugs and edge cases in Claude Code’s permission system that affect WebFetch and WebSearch specifically.

The Problem with Standard Permission Settings

The natural approach is to add WebFetch and WebSearch to your permission configuration:

```
{
  "permissions": {
    "allow": [
      "WebFetch",
      "WebSearch"
    ]
  }
}
```

This should work. Sometimes it does. Often it doesn’t.

1. Wildcard Patterns Don’t Match

Search

Search

RECENT POSTS

- » [What is Microsoft Planner and who is it for](#)
- » [You need to know what the tilde \(~\) does in Excel](#)
- » [To harness crypto ingenuity, financial threat must first be neutralised](#)
- » [Ethereum-Solidity Quiz Q5: What is the Private Mempool?](#)
- » [HOW TO CREATE AN ELASTIC BEANSTALK ON AWS](#)

RECENT COMMENTS

💬 [admin](#) on [What is Microsoft Planner and who is it for](#)

ARCHIVES

- 📅 [December 2025](#)
- 📅 [November 2025](#)
- 📅 [October 2025](#)
- 📅 [July 2025](#)
- 📅 [June 2025](#)

Adding `WebFetch(domain:*)` to allow all domains doesn't actually work . Claude Code still prompts for each new domain, then adds specific entries like `WebFetch(domain:github.com)` to your settings file. The wildcard sits there, ignored.

2. VSCode Extension Ignores Settings

The VSCode extension requires two settings to work together, and the documentation doesn't make this clear:

```
{
  "claudeCode.allowDangerouslySkipPermissions": true,
  "claudeCode.initialPermissionMode": "bypassPermission"
}
```

Without both, your project-level settings are ignored.

3. Sub-Agents Get Auto-Denied

Planning mode and sub-agents (like Explore) run in "dontAsk" mode, which auto-denies tools even when you've configured bypass permissions. You see errors like:

```
Error: Permission to use WebSearch has been auto-denied
```

4. Path Normalization Issues

On macOS and Linux, path expansion inconsistencies cause permission rules to not match. A rule for `/home/username/project/**` won't match a request for `~/project/file.txt` .

The Solution: Use Hooks

Hooks bypass the permission system entirely. They intercept tool calls and return explicit allow/deny decisions before the permission check occurs. This works reliably because:

Two hooks handle different scenarios:

Configuration

Step 1: Create the Hook Script

	May 2025
	April 2025
	March 2025
	January 2025
	December 2024
	October 2024
	September 2024
	August 2024
	June 2024
	May 2024
	April 2024
	March 2024
	February 2024
	January 2024
	December 2023
	November 2023
	June 2023
	May 2023
	April 2023
	December 2022
	October 2022
	July 2022
	June 2022
	May 2022
	March 2022
	December 2021
	April 2020
	September 2019
	August 2019
	May 2019

```
#!/usr/bin/env python3
"""
```

Claude Code hook to auto-approve WebFetch and WebSearch

Exit codes:

0 = Hook made a decision (JSON output printed to stdout)

1 = Hook did not make a decision (fall back to normal Claude Code)

This hook is FAIL-SAFE: any unexpected error exits with code 1
which causes Claude Code to fall back to its normal permissions.
"""

```
import json
import sys
from urllib.parse import urlparse
```

```
# Optional: restrict WebFetch to a domain allowlist.
# Leave as None to allow ALL domains.
ALLOWED_FETCH_DOMAINS = None # e.g. {"learn.microsoft.com"}
```

```
# Optional: force WebSearch to only return results from specific domains.
# Leave as None to not force domain filtering.
FORCE_SEARCH_ALLOWED_DOMAINS = None # e.g. ["learn.microsoft.com"]
```

```
def log_error(msg: str) -> None:
    """Log to stderr for debugging (won't affect hook execution)"""
    print(f"[auto_allow_web_tools] {msg}", file=sys.stderr)
```

```
def host_of(url: str) -> str:
    """Extract lowercase hostname from URL, or empty string if invalid.
    try:
        return (urlparse(url).hostname or "").lower()
    except Exception:
        return ""
```

```
def output_and_exit(result: dict) -> None:
    """
    Print JSON result to stdout and exit with code 0.
    Validates output before printing to prevent malformed JSON.
    """
    try:
```

February 2019

October 2018

September 2018

December 2017

CATEGORIES

business

entertainment

general

health

politics

science

sports

technology

```

        output = json.dumps(result)
        print(output)
        sys.exit(0)
    except (TypeError, ValueError) as e:
        log_error(f"Failed to serialize output: {e}")
        sys.exit(1)

def handle_pre_tool_use(tool: str, tool_input: dict) -> dict:
    """Handle PreToolUse event - decide whether to allow or deny tool use
    updated_input = None

    # Check domain allowlist for WebFetch
    if tool == "WebFetch" and ALLOWED_FETCH_DOMAINS is not None:
        url = tool_input.get("url", "")
        host = host_of(url)
        if host not in ALLOWED_FETCH_DOMAINS:
            output_and_exit({
                "hookSpecificOutput": {
                    "hookEventName": "PreToolUse",
                    "permissionDecision": "deny",
                    "permissionDecisionReason": f"WebFetch domain {host} not in allowlist"
                }
            })
            return # Never reached, but makes flow clear

    # Force domain filtering for WebSearch
    if tool == "WebSearch" and FORCE_SEARCH_ALLOWED_DOMAINS is not None:
        updated_input = dict(tool_input)
        updated_input["allowed_domains"] = list(FORCE_SEARCH_ALLOWED_DOMAINS)

    # Allow the tool call
    result = {
        "hookSpecificOutput": {
            "hookEventName": "PreToolUse",
            "permissionDecision": "allow",
            "permissionDecisionReason": "Auto-approved"
        }
    }
    if updated_input is not None:
        result["hookSpecificOutput"]["updatedInput"] = updated_input

    output_and_exit(result)

def handle_permission_request() -> None:

```

```

"""Handle PermissionRequest event - auto-allow if l
output_and_exit({
    "hookSpecificOutput": {
        "hookEventName": "PermissionRequest",
        "decision": {"behavior": "allow"},
    }
})

```

```
def main() -> None:
```

```

    """

```

```

    Main entry point. Reads hook input from stdin and c

```

```

    IMPORTANT: This hook ONLY handles WebFetch and WebS
    All other tools exit with code 1 (no decision), fa
    """

```

```

    try:

```

```

        # Parse input from Claude Code

```

```

        data = json.load(sys.stdin)

```

```

    except json.JSONDecodeError as e:

```

```

        log_error(f"Invalid JSON input: {e}")

```

```

        sys.exit(1)

```

```

    except Exception as e:

```

```

        log_error(f"Failed to read stdin: {e}")

```

```

        sys.exit(1)

```

```

    event = data.get("hook_event_name", "")

```

```

    tool = data.get("tool_name", "")

```

```

    tool_input = data.get("tool_input") or {}

```

```

    # SECURITY: Only handle WebFetch and WebSearch - no

```

```

    if tool not in ("WebFetch", "WebSearch"):

```

```

        sys.exit(1)

```

```

    # Dispatch to appropriate handler

```

```

    if event == "PreToolUse":

```

```

        handle_pre_tool_use(tool, tool_input)

```

```

    elif event == "PermissionRequest":

```

```

        handle_permission_request()

```

```

    else:

```

```

        # Unknown event type - don't make a decision

```

```

        log_error(f"Unknown event type: {event}")

```

```

        sys.exit(1)

```

```

if __name__ == "__main__":

```

```

try:
    main()
except Exception as e:
    # Catch-all: ensure we NEVER exit 0 without valid output
    log_error(f"Unexpected error: {e}")
    sys.exit(1)

```

Step 2: Configure the Hook

Add the hook configuration to `.claude/settings.local.json` :

```

{
  "permissions": {
    "allow": [],
    "deny": [],
    "ask": []
  },
  "hooks": {
    "PreToolUse": [
      {
        "matcher": "WebFetch|WebSearch",
        "hooks": [
          {
            "type": "command",
            "command": "python3 .claude/hooks/auto_allow.py",
            "timeout": 5
          }
        ]
      }
    ],
    "PermissionRequest": [
      {
        "matcher": "WebFetch|WebSearch",
        "hooks": [
          {
            "type": "command",
            "command": "python3 .claude/hooks/auto_allow.py",
            "timeout": 5
          }
        ]
      }
    ]
  }
}

```

Windows users : Replace python3 with py -3 or the full path to your Python executable.

How It Works

PreToolUse Hook

When Claude decides to call WebFetch or WebSearch:

PermissionRequest Hook

If a permission dialog would appear (edge cases, sub-agents):

Fail-Safe Design

The script exits with code 1 for any unexpected condition:

Optional: Domain Restrictions

The script supports optional domain filtering. Uncomment and configure:

```
# Only allow fetching from these domains
ALLOWED_FETCH_DOMAINS = {"docs.microsoft.com", "github.com"}

# Force all web searches to only return results from the allowed domains
FORCE_SEARCH_ALLOWED_DOMAINS = ["docs.microsoft.com", "github.com"]
```

With ALLOWED_FETCH_DOMAINS set, WebFetch requests to unlisted domains are denied with a clear message.

With FORCE_SEARCH_ALLOWED_DOMAINS set, the hook modifies the WebSearch input to include domain filtering, regardless of what Claude requested.

Project vs Global Configuration

You can place this configuration in different locations:

For team projects, commit .claude/hooks/auto_allow_web_tools.py but keep settings.local.json in .gitignore so each developer can configure their own preferences.

Debugging

If the hook isn't working:

Check the script runs : echo

```
'{"hook_event_name":"PreToolUse","tool_name":"WebFetch","tool_input":  
{ "url":"https://example.com"} }' | python3 .claude/hooks/auto_allow_web_tools.py
```

Check stderr : Errors are logged to stderr, visible in Claude Code’s debug output

Verify the matcher : The regex WebFetch|WebSearch must match exactly

Check Python path : Ensure python3 (or py -3 on Windows) is available in PATH

Summary

Hooks work where permission settings don’t. The exit-code-1 fallback means you’re never locked out if something goes wrong.

Learn more:

Have questions or improvements? Open an issue or PR on your project’s repository.



Templates let you quickly answer FAQs or store snippets for re-use.

Are you sure you want to hide this comment? It will become hidden in your post, but will still be visible via the comment’s permalink .

Hide child comments as well

Confirm

For further actions, you may consider blocking this person and/or reporting abuse

 Tagged

AI

coding

community

development

engineering

inclusive

Productivity

Programming

software

tutorial

RELATED POSTS

2025.8 Release
Introduces Stack
Overflow Internal: The
Next Generation Of
Enterprise Knowledge
Intelligence

November 12, 2025 
admin

Test Your Drupal Website
With Cypress

April 26, 2023 
admin

Microsoft’s Framework
For Building AI Systems
Responsibly

June 21, 2022 
admin

Leave a Reply

Your email address will not be published. Required fields are marked *

Comment *

Name *

Email *

Website

☐
Save my name,
email, and website
in this browser for
the next time I
comment.

Post Comment

